

AquaSavannah LandVault

Combined Planning Document

Part A: v3 Historical Snapshot (Superseded) · Part B: v4 Current Authoritative Plan

Build against Part B only — see Preface

Repository: github.com/chuzcltdchuzc-gif/Development-Plan
Derived from full architecture & security audits of the Base44 (landsecure-registry)
and Emergent (landverify-nigeria-101) codebases

- Preface
- Part A — v3 Historical Snapshot
- v3 Rebuild Plan (Superseded)
- Part B — v4 Current Plan
- Project Overview
- Claude Code Operating Pointer
- Rebuild Plan (v4 — current)
- Phase Gates & Process Model
- Definition of Done
- Engineering Operating Rules
- ADR-001
- ADR-002
- ADR-003
- ADR-004
- ADR-005
- ADR-006
- ADR-007
- ADR-008

Preface — How to Read This Document

This PDF combines two planning artifacts produced during the AquaSavannah LandVault architecture and migration audit, at the Operator's request, into a single reference document.

Part A (following this preface) is the **v3 Rebuild Plan** — a snapshot from earlier in the planning process, generated before the final stack decisions were locked in. It targeted **React + Vite** for the frontend and a fully self-issued JWT authentication scheme, with no explicit statement that the PDP/PEP authorization engine would be retained. It is preserved here **for provenance only** — it shows the reasoning trail that led to the current architecture, consistent with this project's own ADR discipline of recording *why* decisions changed, not just *what* they are now.

Part B is the **current, authoritative v4 planning package** — the full governance and architecture set now committed to the **Development-Plan** repository (**README.md**, **CLAUDE.md**, **docs/REBUILD_PLAN.md**, **docs/PHASE_GATES.md**, **docs/DOD.md**, **docs/ENGINEERING_RULES.md**, and ADR-001 through ADR-008). It reflects the final stack: **Next.js + TypeScript** (frontend), **FastAPI** (backend), **PostgreSQL + PostGIS** (database), **Keycloak/Auth0** for authentication with the **PDP/PEP/PIP** authorization engine explicitly retained, **S3-compatible object storage**, **Paystack + Stripe** payments, and **Docker + Terraform + AWS/Azure** infrastructure.

If Part A and Part B ever appear to disagree on any technical detail — stack choice, effort estimate, or scope — Part B is correct. Part A is historical record, not a build target. Any Claude Code session working from this document should treat **docs/REBUILD_PLAN.md** (reproduced in Part B) as the single source of truth for what to build.

Part A — v3 Rebuild Plan (Historical Snapshot, Superseded)

Generated earlier in the planning process. See Preface. Superseded in full by Part B.

AquaSavannah LandVault — Rebuild Plan (Backend / Frontend / Feature tracks)

v3 — Survey confirmed as bounded context #13, built directly off Base44's existing Surveyor Network / SurveyorDashboard / ArchiveRecord module (schema + UX kept as-is; backend logic rebuilt since none of substance existed to inherit — the module sat at 0 live records in the audited data). Previous revision (v2) added Spatial Intelligence, Trust Engine, and Knowledge Graph and inserted milestone M2.5.

Source material: full audits of `landsecure-registry` (Base44 build) and `landverify-nigeria-101-NEW` (Emergent build), both completed earlier in this session.

0. Target stack decision (unchanged)

Layer	Decision	Why
Backend	Python / FastAPI, DDD/hexagonal layering	Emergent's bounded-context code is genuinely well-designed where the parallel legacy layer isn't undermining it
Database	PostgreSQL (+ PostGIS for Spatial Intelligence) for Identity/Registry/Spatial/Survey/Economic. Object storage (S3/R2, Object Lock Compliance mode) for Evidence binaries. Dedicated graph store (Neo4j or Postgres+AGE) for Knowledge Graph, fed by event projection	Closes the ACID/RLS bug class both audits kept surfacing; PostGIS for real overlap math; graph store for real multi-hop traversal
Authorization	One JWT + PDP/PEP/ABAC engine, applied everywhere, no legacy bypass layer	Fixes the dual-auth-system defect found in the Emergent audit
Storage/Evidence	Real WORM: S3/R2 Object Lock from day one	Both prior "WORM" implementations were fake at the storage layer
Frontend	React + Vite + TypeScript + Tailwind + shadcn/ui, one generated client from a frozen OpenAPI contract	Both prior apps independently converged on this stack
Sessions	httpOnly/Secure/SameSite=Lax cookies only, never localStorage	Fixes the Emergent frontend's XSS-exfiltration finding
Trust Engine transport	Event-driven off the same transactional outbox every context publishes to	No new integration mechanism needed
Knowledge Graph transport	Read-model projection off the outbox (reuse Emergent's projection-engine pattern)	Keeps the graph rebuildable/derived, never authoritative

1. The 13 bounded contexts

#	Context	Responsibilities	Source of truth for design
1	Identity	Users, hierarchy-checked roles, sessions, JWT, delegation, service accounts	New build, pattern from Emergent
2	Registry	Canonical parcel aggregate, ownership history, parcel numbering	Emergent invariants + Base44 field model
3	Spatial Intelligence	Coordinate/polygon validation, overlap + duplicate-geometry detection, GIS indexing, spatial search, adjacency, distance, map tiling, satellite overlays, spatial analytics	New build (server-side), Base44's <code>spatialValidation.js</code> as a starting reference
4	Evidence	Upload, server-side hashing, WORM sealing, Merkle anchoring, chain of custody, enforced legal hold	Emergent's pipeline (audited sound)
5	Survey (confirmed)	Surveyor licensing/credentials, assignment, survey plan upload (through the Evidence pipeline), archive import, revenue-share events	Base44's Surveyor Network / SurveyorDashboard / ArchiveRecord module, kept as-is (schema + UX), logic rebuilt
6	Trust Engine	Continuously recalculated, explainable per-parcel trust score, aggregating signals from every context below as they land	New build; <code>subscores</code> breakdown shape borrowed from Base44's <code>lvTrustValidationEngine</code> (structure only, not the math)
7	Workflow	Generic state-machine engine, sagas, timers, SLA/escalation, compensation, with real command handlers	Emergent's engine wholesale
8	Community Trust	Attestation, honest consensus scoring, conflict detection, traditional-authority endorsement	Base44's domain/field model as spec
9	Inheritance & Customary Law	Death verification, beneficiary/family ownership, regime selection, share calculation, disputes	Base44 + Emergent domain models as spec (neither implemented securely)
10	Economic / Billing	Credit wallet (atomic), service catalog, invoicing, payments, surveyor revenue-share consumption	Base44 field model + Emergent's audited-correct webhook code
11	Knowledge Graph	Surveyor→Parcel→Community→Family→Inheritance→Verification→Evidence→Economic→Certificates→Disputes→Banks→Lawyers→Government	New build

#	Context	Responsibilities	Source of truth for design
12	Security	Fraud detection, security incidents, real permission auditing, pen-test harness	Both apps' module lists as spec
13	Operations	Background job queue, backup/recovery testing, deployment observability	Both apps' job-type catalogs as spec

2. BACKEND build plan

Stage	Scope	Reuse from prior builds	Discard	Security fix baked in	Effort
B0 — Kernel foundation	Postgres schema/migrations, fail-closed config, structured logging, RFC-7807 errors, health checks	Emergent's <code>kernel/errors/problem.py</code>	—	No permissive env-var defaults anywhere; CI fails the build if found	1–2 wk
B1 — AuthN/AuthZ engine	JWT+JWKS+rotation w/ real hard-revoke, single PDP/PEP/PIP, audit hash-chain + <code>verify_chain()</code> checker (missing in Emergent), tenant/country scoping via Postgres RLS	Emergent's fail-closed PDP/PEP pattern; sound crypto code	Emergent's legacy session-cookie <code>core/security.py</code> RBAC entirely	Rate limiting on by default; CORS origin list required; <code>X-Forwarded-For</code> only trusted behind a configured proxy allowlist	2–3 wk
B2 — Identity	User mgmt, role assignment with hierarchy check , delegation wired into the PDP	Emergent's user/session/delegation aggregates; refresh-token replay-revocation logic	Broken <code>assign_role</code> as-is; <code>emergent_google</code> -specific OAuth coupling	Self-registration can never set role; every role change audited	2 wk
B3 — Registry	Parcel aggregate, immutable-field invariants, atomic allocator, append-only ownership history	Emergent's <code>_apply_patch</code> guard, atomic allocator; Base44's entity field list as spec	Base44 entity code as-is; Emergent's dual-write legacy adapter	Ownership-transfer authorization checks real actor identity against the resource	3–4 wk
B4 — Spatial Intelligence	PostGIS-backed validation, overlap + duplicate-geometry detection, spatial indexing/search, adjacency, distance, map tiling	Base44's <code>spatialValidation.js</code> logic as reference (move server-side, make authoritative)	Base44's hardcoded LGA bounding-box check — replace with real polygon containment	Duplicate-geometry becomes a real signal (not GPS-proximity-only); first Trust Engine signal source	3–4 wk
B5 — Evidence	Upload → server-side hash (stream, independent read-back) → real S3/R2 Object Lock seal → Merkle anchoring → chain of custody → enforced legal hold	Emergent's hash/verify pipeline, Merkle/CT-log/OTS saga pattern	Chmod-based fake WORM; never-finished R2 stub	Legal hold as a guard at every delete/archive/seal-release path; break-glass dual-auth mechanically enforced	4–6 wk
B6 — Survey <i>(confirmed, kept-as-is)</i>	Surveyor onboarding/licensing, assignment-to-parcel workflow, survey plan upload through the Evidence pipeline (real hash/seal, not plain file upload), archive import, revenue-share event emission	<code>SurveyorPartner</code> , <code>SurveyAssignment</code> , <code>ArchiveRecord</code> , <code>RevenueTransaction</code> , <code>SurveyDocument</code> schemas kept verbatim ; consolidates Base44's split legacy/LandVault survey tracks into one module	Base44's dual legacy-vs-LandVault duplication (two parallel page sets for the same concept)	Assignment/completion actions require real role + licence-status checks server-side (previously client-side-only); survey plans get the same integrity guarantees as any other evidence	2–3 wk
B7 — Trust Engine	Event-subscribing scoring engine; scaffolds with 3 signals (Evidence integrity + Spatial duplicate-geometry + Survey completion/licence-confidence), explainable score breakdown API, continuously recalculated	Base44's <code>subscores</code> object shape (structure only)	All of Base44's actual scoring functions (the "always passes" defect)	CI rule: every signal contributor must have a test asserting zero-data input yields <code>INSUFFICIENT_DATA</code> , never a high score	2–3 wk (scaffold) + incremental per later stage
B8 — Workflow engine	Real command handlers wired to Registry/Evidence/Identity/Survey	Emergent's engine wholesale (audited as the most rigorously built part of that codebase)	<code>NullCommandHandler</code>	Compensation failures alert, don't silently log-and-continue	2–3 wk
B9 — Community Trust	Attestation, honest consensus scoring, conflict detection, traditional-authority endorsement — 4th Trust Engine signal	Base44's domain/field model as spec	Base44's scoring code outright	Real authenticated actor on every review	2–3 wk
B10 — Inheritance & Customary Law	Death verification → beneficiary validation → share calculation → certificate — feeds Trust Engine identity-confidence, feeds Knowledge Graph	Base44's + Emergent's domain models as spec	Both apps' actual implementations	PII routes through B5's encryption from day one	4–5 wk
B11 — Economic / Billing	Atomic ledger, real invoicing, payment webhooks, consumes Survey's revenue-share events — feeds Knowledge Graph's economic trees	Emergent's webhook signature code (audited correct); Base44's <code>RevenueTransaction</code> field model	Mock payment endpoint entirely; Base44's billing crash/race bugs	Balance mutation only via one atomic ledger function, DB-constrained non-negative	3 wk
B12 — Knowledge Graph	Graph projection off the event outbox across all contexts including Survey (surveyor↔parcel edges)	New build	—	Read-only, rebuildable from the outbox at any time	3–4 wk
B13 — Security	Real fraud detection (queries the Knowledge Graph), security incidents, permission auditor reading actual RLS policy config , pen-test harness	Both apps' module lists as spec	Base44's fictional shadow-table permission auditor; Emergent's "R-2 CLOSED" self-cert pattern	Permission auditor fails loudly on any table with missing/permissive policy	2–3 wk
B14 — Operations	Job queue, backup/recovery testing, deployment observability	Both apps' job-type catalogs as spec	Base44's disabled-by-default automations; unconditional <code>seed_demo_data()</code> -style startup seeding	All seeding environment-gated, never runs against a populated/production DB automatically	2 wk

Backend total: ~33–40 weeks sequential (up from ~30–37 in v2, reflecting Survey's addition — offset somewhat by it needing very little net-new domain design work since the schema/UX are inherited).

3. FRONTEND build plan

Stage	Scope	Reuse	Discard	Security fix	Effort
F0 Shell	Design system, layout, generated client, <code>ProtectedRoute</code> on every route	Base44's shadcn/ui set + layout pattern; Emergent's SDK discipline	Both apps' auth-context implementations	Route-audit lint rule	1–2 wk
F1 Auth	Login/register/OIDC with real CSRF state	Base44's role-selection UX	Emergent's stateless OAuth callback	CSRF-protected callback	1 wk
F2 Registry + Spatial UI	Parcel CRUD, GIS map, polygon editor, overlap/duplicate warnings live during registration, bulk import	Base44's map/form components as reference	Naive CSV parser; client-side-only LGA bounds check	Spatial validation calls B4 server-side, never trusts client geometry	3–4 wk
F3 Evidence UI	Upload/list/detail/tabs	Emergent's evidence pages wholesale	—	Missing <code>ProtectedRoute</code> added; SDK path-param encoding fixed	2 wk
F3.5 Survey / Surveyor Network UI (confirmed, kept-as-is)	Surveyor dashboard, surveyor network directory, public surveyor profile, archive import wizard	<code>SurveyorDashboard.jsx</code> , <code>SurveyorNetwork.jsx</code> , <code>SurveyorPublicProfile.jsx</code> , <code>ArchiveImportWizard.jsx</code> ported closely — same layout/UX, wired to the new secure SDK	Base44's separate legacy-tier survey pages (<code>SurveyDocuments.jsx</code> , <code>SurveyReviews.jsx</code> , <code>AssignedParcels.jsx</code>) — consolidated into this one module	Assignment actions require server-verified role/licence status, not just UI gating	2–3 wk
F3.6 Trust & Transparency UI	Trust dashboard with explainable score breakdown , duplicate-alert dashboard, evidence timeline, public transparency portal	Base44's <code>TrustArchitecture.jsx</code> and <code>CommunityTransparency.jsx</code> layout	Base44's single-number trust score display with no factor breakdown	Every displayed score is paired with the signal list that produced it	2–3 wk
F4 Community Trust UI	Attestation/review/consensus	Base44 UX as reference	Hardcoded- <code>"admin"</code> -actor components	Real authenticated actor on every review	2–3 wk
F5 Inheritance UI	Case/beneficiary/certificate mgmt	Base44 UX as reference	<code>document.write()</code> cert generator (confirmed XSS)	No client-built HTML strings anywhere	3–4 wk
F6 Economic UI	Wallet/billing/revenue, surveyor revenue-share view	Base44 pages as reference	Direct wallet/invoice writes	UI confirms only after atomic server write	2–3 wk
F7 Knowledge Graph explorer	Relationship visualization for fraud/due-diligence users	New build	—	Read-only viewer; no mutation surface via the graph	2–3 wk
F8 Security/Ops/Gov dashboards	Admin consoles	Both apps' layouts as reference	Score-without-evidence display pattern	Server-verified role, not sidebar-hiding	2–3 wk
F9 Public pages	Landing, verify, transparency	Both apps' content	—	CI route-audit confirms nothing else is public	1 wk

Frontend total: ~23–29 weeks.

4. FEATURE rollout timeline

Milestone	Unlocks	Needs	Cumulative (parallelized)
M0 Platform online	Auth works, CI green	B0-B2, F0-F1	Weeks 1–4
M1 Core registry MVP	Register/search/verify a parcel, real spatial validation at registration	B3-B4, F2, F9	Weeks 4–12
M2 Evidence-backed registry	Real evidence, integrity, enforced legal hold	B5, F3	Weeks 10–18
M2.4 Surveyor Network live	Surveyor onboarding, assignment, archive import, survey plans through the Evidence pipeline	B6, F3.5	Weeks 16–20
M2.5 Trust Intelligence	Confidence engine, duplicate detection (spatial + hash), chain of custody, trust dashboard with explainable breakdown, transparency dashboard, evidence timeline — now backed by 3 real signals (Evidence + Spatial + Survey)	B7, F3.6	Weeks 18–24
M3 Community trust	Attestations feed Trust Engine as a 4th signal; consensus, traditional authority	B8-B9, F4	Weeks 22–28
M4 Inheritance & customary law	Death→beneficiary→share→certificate, PII-protected, feeds identity-confidence + Knowledge Graph	B10, F5	Weeks 26–34
M5 Economic operations	Real billing, atomic wallets, surveyor revenue-share paid out, feeds Knowledge Graph	B11, F6	Weeks 30–36
M6 Knowledge Graph & fraud intelligence	Relationship graph live, fraud detection consumes it	B12-B13, F7-F8	Weeks 34–40
M7 Operations hardening & pilot launch	Real job queue, backup/recovery tested, environment-gated seeding, runbook	B14, F9 polish	Weeks 38–42

Total to pilot-ready: ~40–42 weeks.

5. Maturity read: Base44 vs. M2.5 + M2.4 deliverables

(Unchanged from v2 for the original M2.5 items; Survey added.)

Deliverable	Base44 state	Read
Surveyor Network / Dashboard / Archive Import (M2.4)	Fully scaffolded pages + entities, routed, but 0 live records across <code>SurveyorPartner</code> , <code>SurveyAssignment</code> , <code>ArchiveRecord</code> , <code>RevenueTransaction</code> — never actually exercised	~70% reusable as UX/schema (this is the highest-fidelity reuse case in the whole plan, precisely because so little broken logic was ever built on top of it), ~0% backend logic to inherit since there wasn't much there
Confidence Engine	<code>lvEvidenceConfidence</code> / <code>lvAttestationConfidence</code> exist, inverted <code>risk_score</code> , unauthenticated write	~50% spec, 0% code
Duplicate Detection	GPS-proximity only, 7 redundant automations, 0 alerts ever generated	~40% spec
Chain of Custody	Functionally real (10 records confirmed working), <code>EvidenceChain</code> entity has no RLS at all	~55% UX, needs full security rebuild
Trust Dashboard	Real, well-built UI; underlying number is the confirmed "100/A_PLUS/GO despite 0 evidence" defect	~60% UI, ~10% trustworthy data
Transparency Dashboard	Confirmed working with real data (8 attestations)	~65% directly reusable
Explainable Trust Score	<code>subscores</code> breakdown shape exists structurally; every value inside is fabricated by no-op branches	~15%

6. Open questions remaining

- 1. **Knowledge Graph engine** — Neo4j (separate service, more capable) vs. Postgres + Apache AGE (one database, less mature tooling)? Leaning AGE for a lean team, Neo4j if fraud/lineage analytics becomes a serious product surface.
- 2. Team size — still assuming 1–3 engineers + Claude Code; estimates rescale roughly linearly if that's wrong.

Part B — v4 Current Authoritative Planning Package

This is the live content of the [Development-Plan](#) GitHub repository. Build against this.

AquaSavannah LandVault

A land-registry and verification platform for Nigeria — parcel registration, GIS-backed spatial validation, evidence chain-of-custody, community/traditional-authority attestation, survey network management, inheritance & customary-law resolution, and an economic/billing layer, built around an explainable, continuously-recalculated per-parcel trust score.

Status: Planning — Architecture v1.0 not yet frozen. No application code has been written yet. This repository currently holds the full governance/planning package produced from a from-scratch architecture and security audit of two prior implementations of this product.

Why a rebuild

Two prior builds of this product were audited in full (architecture, security, correctness) before any code here was written — see [docs/audits/](#):

- **Base44 build** — full-featured but insecure: client-side-only authorization throughout, permissive/missing row-level-security on numerous entities (including a self-service financial-fraud vector on user wallets), and multiple "trust validation" functions confirmed to report a passing score regardless of actual data.
- **Emergent build** — a genuinely well-architected DDD/event-sourced backend (its authorization engine design is retained in this rebuild — see ADR-004) undermined by a parallel, undocumented legacy auth system and an unauthenticated admin-login bypass.

Every rule in [docs/ENGINEERING_RULES.md](#) and every non-negotiable in [docs/DOD.md](#) traces to a specific, confirmed finding from these audits — this isn't generic best-practice boilerplate.

Documentation map

Document	What it's for
CLAUDE.md	How Claude Code should operate in this repo — read this first
docs/REBUILD_PLAN.md	The technical plan: target stack, 13 bounded contexts, backend/frontend build stages, feature-rollout milestones
docs/PHASE_GATES.md	The process/quality-gate model (Phase 0–12), the Claude Code Loop, and the 10 standing review questions
docs/DOD.md	Definition of Done — Feature / Sprint / Product (MVP) tiers
docs/ENGINEERING_RULES.md	When Claude may act autonomously vs. must stop and ask; the non-negotiable engineering rules
docs/adr/	Architecture Decision Records — ADR-001 through ADR-008
docs/audits/	The full audit reports behind every decision above

Stack

Layer	Choice
Frontend	Next.js + TypeScript, Tailwind + shadcn/ui
Backend	Python + FastAPI, DDD/hexagonal architecture
Database	PostgreSQL + PostGIS
Authentication	Keycloak (default) or Auth0
Authorization	Custom PDP/PEP/PIP engine (retained from audit-validated Emergent design)
Object storage	S3-compatible, Object Lock (Compliance mode)
Payments	Paystack (Nigeria) + Stripe (diaspora)
Infrastructure	Docker, Terraform, AWS/Azure
Audit	Event sourcing + immutable hash-chained logs

Full rationale for each choice: [docs/adr/](#).

Process at a glance

Every phase must pass its quality gates before the next one starts — see [docs/PHASE_GATES.md](#) for full detail.

Phase	Exit criterion
0 — Enterprise Planning	Vision, requirements, and scope approved and frozen
1 — System Architecture	No unresolved architecture issues
2 — Development Environment	Three isolated environments operational
3 — Foundation	Zero critical vulnerabilities
4 — Database	Database production ready
5 — Core Services	95% automated test coverage

Phase	Exit criterion
6 — AI Layer	AI quality above target threshold
7 — Payments	Successful end-to-end payment tests
8 — Security Hardening	No critical or high-severity findings
9 — Performance	Performance targets achieved
10 — Production Readiness	Go/No-Go decision
11 — Launch	Live, monitored, reviewed
12 — Growth	Ongoing

Sprints are organized **one per bounded context** (13 total — see `docs/REBUILD_PLAN.md` §1), not by generic feature grouping.

Quick start

No runnable code yet — this repository is currently documentation-only, at the close of Phase 0/1. Setup instructions land here at Phase 2 (Development Environment).

License

TBD.

CLAUDE.md

AquaSavannah LandVault — a Nigerian land-registry/verification platform, rebuilt from scratch on Claude Code after full security/architecture audits of two prior implementations (`docs/audits/`). **Current status: Phase 0/1 (Enterprise Planning / System Architecture) — planning package complete, Architecture v1.0 not yet frozen. No implementation sprint has started.**

This file is the always-loaded operational summary. It is a pointer, not the source of truth — if anything here ever conflicts with the documents it points to, **those documents win**.

The 5 non-negotiable rules (full detail: `docs/ENGINEERING_RULES.md`)

- 1. **No new entity/table without an RLS/authorization policy in the same commit.** (Base44 shipped wallet/invoice entities with unconditional public update access — this is the exact bug class that rule prevents.)
- 2. **No permissive fallback default on any security-relevant env var.** Missing config must fail startup, never silently degrade to an insecure default. (Emergent's CORS wildcard-with-credentials and hardcoded signing-secret fallback.)
- 3. **Exactly one authorization path: the PDP/PEP/PIP engine.** No parallel/legacy auth system, no unguarded dev-login, ever — not even temporarily. (Emergent's dual auth system + unauthenticated admin bypass.)
- 4. **Every scoring/validation function fails safe:** zero/missing data → low or neutral result, never a passing score. (Base44's trust engine reported 100/A+ with zero real evidence.)
- 5. **Never mark something complete without having actually observed it pass.** Static code inspection is not evidence — run the test, see it pass.

Where to look for more

Need	Go to
The technical build plan (stack, 13 bounded contexts, stages, milestones)	<code>docs/REBUILD_PLAN.md</code>
Process/quality gates per phase, the Claude Code Loop, standing review questions	<code>docs/PHASE_GATES.md</code>
Definition of Done (Feature / Sprint / Product tiers)	<code>docs/DOD.md</code>
Full engineering rules, incl. when to stop and ask a human	<code>docs/ENGINEERING_RULES.md</code>
Why a specific architectural decision was made	<code>docs/adr/</code>
The audit findings everything above is derived from	<code>docs/audits/</code>

Repo layout (once code lands)

```
/frontend  - Next.js + TypeScript
/backend   - Python + FastAPI, one folder per bounded context
/infra     - Terraform, Docker
/docs      - this planning package
```

Working model

Sprints are one per bounded context (13 total, dependency-ordered per `docs/REBUILD_PLAN.md` §1), each gated through the Claude Code Loop in `docs/PHASE_GATES.md` and signed off against `docs/DOD.md` before merge. Do not start a sprint whose dependencies (per the bounded-context ordering) aren't yet Sprint Done.

Build/test/run commands land here once Phase 2 (Development Environment) is complete.

AquaSavannah LandVault — Rebuild Plan (Backend / Frontend / Feature tracks)

v4 — migrated to disk from planning conversation; stack updated to Next.js (frontend), Keycloak/Auth0 (identity, PDP/PEP/PIP retained), Docker+Terraform+AWS/Azure (infrastructure), per Operator decision. Sprints are now formally one-per-bounded-context — see [docs/PHASE_GATES.md](#) for how the Phase 0–12 gate model governs progression between them.

Source material: full architecture/security audits of [landsecure-registry](#) (Base44 build) and [landverify-nigeria-101-NEW](#) (Emergent build) — see [docs/audits/](#).

0. Target stack decision

Layer	Decision	Why
Frontend	Next.js + TypeScript (App Router), Tailwind + shadcn/ui	Reuses the shadcn/ui component set both prior apps independently converged on; App Router gives file-based routing and room for SSR on public verification pages later
Backend	Python / FastAPI , DDD/hexagonal layering (domain → ports → adapters → application → api)	Emergent's bounded-context code is genuinely well-designed where it isn't undermined by its parallel legacy layer
Database	PostgreSQL + PostGIS (Identity role/tenant mapping, Registry, Spatial, Survey, Economic), self-managed (not Supabase-managed)	Nearly every critical bug in both audits traced back to not having real ACID transactions and native RLS — Postgres closes that bug class structurally
Object storage	S3-compatible, Object Lock in Compliance mode , for Evidence binaries	Both prior "WORM" implementations were fake at the storage layer (chmod meaningless under root; ephemeral <code>/tmp</code> by default)
Identity (authentication)	Keycloak (default) or Auth0 , issuing OIDC/JWT identity tokens	Removes an entire class of custom-auth bugs found in both audits (password handling, MFA, OAuth flows, key rotation) by delegating to a proven IdP instead of reimplementing it
Authorization	PDP/PEP/PIP engine retained , ported from Emergent's design — verifies IdP-issued JWTs against the IdP's JWKS (not a self-issued KeyStore), applies fail-closed policy decisions to every route, no parallel/legacy auth path	This was the one subsystem the Emergent audit found structurally sound — the defect was letting a second, undocumented session-cookie auth system exist beside it. Removing the parallel system and keeping this one fixes the root cause
Graph store (Knowledge Graph)	Neo4j or Postgres + Apache AGE, fed by event projection	Real multi-hop relationship queries; never a source of truth, always rebuildable
Payments	Paystack (Nigeria) + Stripe (diaspora)	Matches audited-correct webhook signature verification code from Emergent, reused near-verbatim
Infrastructure	Docker (local + CI), Terraform (all cloud resources as code), AWS or Azure	Repeatable environments across dev/staging/production; closes the "works on my machine" and untracked-manual-config gaps found in both audits
Sessions	httpOnly, Secure, SameSite=Lax cookies only. Never a token in <code>localStorage</code>	Direct fix for the Emergent frontend's XSS-exfiltration finding
Audit	Event sourcing + transactional outbox + hash-chained immutable audit log, with a real <code>verify_chain()</code> integrity checker (missing in Emergent)	Reuses Emergent's outbox/audit design, closes its one gap
CI/tests	Real test suite + working CI from commit #1	Base44's CI was broken from day one (no test script, doomed <code>npm publish</code> workflow)

1. The 13 bounded contexts

#	Context	Responsibilities	Source of truth for design
1	Identity	User→IdP mapping, hierarchy-checked role assignment, tenant scoping, delegation, service accounts	New build; Keycloak/Auth0 for authentication, custom for authorization data
2	Registry	Canonical parcel aggregate, ownership history, parcel numbering	Emergent invariants + Base44 field model
3	Spatial Intelligence	Coordinate/polygon validation, overlap + duplicate-geometry detection, GIS indexing, spatial search, adjacency, distance, map tiling, satellite overlays, spatial analytics	New build (server-side, PostGIS), Base44's <code>spatialValidation.js</code> as a starting reference
4	Evidence	Upload, server-side hashing, WORM sealing, Merkle anchoring, chain of custody, enforced legal hold	Emergent's pipeline (audited sound)
5	Survey	Surveyor licensing/credentials, assignment, survey plan upload (through the Evidence pipeline), archive import, revenue-share events	Base44's Surveyor Network / SurveyorDashboard / ArchiveRecord module, kept as-is (schema + UX), logic rebuilt
6	Trust Engine	Continuously recalculated, explainable per-parcel trust score, aggregating signals from every context below as they land	New build; <code>subscores</code> breakdown shape borrowed from Base44's <code>lvTrustValidationEngine</code> (structure only, not the math)
7	Workflow	Generic state-machine engine, sagas, timers, SLA/escalation, compensation, with real command handlers	Emergent's engine wholesale
8	Community Trust	Attestation, honest consensus scoring, conflict detection, traditional-authority endorsement	Base44's domain/field model as spec
9	Inheritance & Customary Law	Death verification, beneficiary/family ownership, regime selection, share calculation, disputes	Base44 + Emergent domain models as spec (neither implemented securely)

#	Context	Responsibilities	Source of truth for design
10	Economic / Billing	Credit wallet (atomic), service catalog, invoicing, payments, surveyor revenue-share consumption	Base44 field model + Emergent's audited-correct webhook code
11	Knowledge Graph	Surveyor→Parcel→Community→Family→Inheritance→Verification→Evidence→Economic→Certificates→Disputes→Banks→Lawyers→Government	New build
12	Security	Fraud detection, security incidents, real permission auditing, pen-test harness	Both apps' module lists as spec
13	Operations	Background job queue, backup/recovery testing, deployment observability	Both apps' job-type catalogs as spec

2. BACKEND build plan

Stage	Scope	Reuse from prior builds	Discard	Security fix baked in	Effort
B0 — Kernel foundation	Postgres schema/migrations (Alembic), fail-closed config, structured logging, RFC-7807 errors, health checks, Docker/Terraform scaffold	Emergent's <code>kernel/errors/problem.py</code>	—	No permissive env-var defaults anywhere; CI fails the build if found	1–2 wk
B1 — AuthN/AuthZ engine	JWT verification against Keycloak/Auth0's JWKS (not self-issued keys), single PDP/PEP/PIP, audit hash-chain + <code>verify_chain()</code> checker, tenant/country scoping via Postgres RLS	Emergent's fail-closed PDP/PEP pattern in full	Emergent's self-issued JWT/KeyStore/rotation code (Keycloak/Auth0 now owns this); Emergent's legacy session-cookie <code>core/security.py</code> RBAC entirely	Rate limiting on by default; CORS origin list required; X-Forwarded-For only trusted behind a configured proxy allowlist	1–2 wk (<i>reduced</i> — <i>issuance delegated to IdP</i>)
B2 — Identity	User→IdP-subject mapping, role assignment with hierarchy check , delegation wired into the PDP, tenant provisioning	Emergent's role/delegation aggregate logic	Broken <code>assign_role</code> as-is; Emergent's password/MFA/OAuth-provider code (not needed — IdP owns this)	Self-registration can never set role; every role change audited	1–2 wk (<i>reduced</i>)
B3 — Registry	Parcel aggregate, immutable-field invariants, atomic allocator, append-only ownership history	Emergent's <code>_apply_patch</code> guard, atomic allocator; Base44's entity field list as spec	Base44 entity code as-is; Emergent's dual-write legacy adapter	Ownership-transfer authorization checks real actor identity against the resource	3–4 wk
B4 — Spatial Intelligence	PostGIS-backed validation, overlap + duplicate-geometry detection, spatial indexing/search, adjacency, distance, map tiling	Base44's <code>spatialValidation.js</code> logic as reference (move server-side, make authoritative)	Base44's hardcoded LGA bounding-box check — replace with real polygon containment	Duplicate-geometry becomes a real signal (not GPS-proximity-only); first Trust Engine signal source	3–4 wk
B5 — Evidence	Upload → server-side hash (stream, independent read-back) → real S3 Object Lock seal → Merkle anchoring → chain of custody → enforced legal hold	Emergent's hash/verify pipeline, Merkle/CT-log/OTS saga pattern	Chmod-based fake WORM; never-finished R2 stub	Legal hold as a guard at every delete/archive/seal-release path; break-glass dual-auth mechanically enforced	4–6 wk
B6 — Survey	Surveyor onboarding/licensing, assignment-to-parcel workflow, survey plan upload through the Evidence pipeline, archive import, revenue-share event emission	<code>SurveyorPartner</code> , <code>SurveyAssignment</code> , <code>ArchiveRecord</code> , <code>RevenueTransaction</code> , <code>SurveyDocument</code> schemas kept verbatim; consolidates Base44's split legacy/LandVault survey tracks into one module	Base44's dual legacy-vs-LandVault duplication	Assignment/completion actions require real role + licence-status checks server-side	2–3 wk
B7 — Trust Engine	Event-subscribing scoring engine; scaffolds with 3 signals (Evidence integrity + Spatial duplicate-geometry + Survey completion/licence-confidence), explainable score breakdown API	Base44's <code>subscores</code> object shape (structure only)	All of Base44's actual scoring functions (the "always passes" defect)	CI rule: every signal contributor must have a test asserting zero-data input yields <code>INSUFFICIENT_DATA</code> , never a high score	2–3 wk + incremental
B8 — Workflow engine	Real command handlers wired to Registry/Evidence/Identity/Survey	Emergent's engine wholesale (audited as the most rigorously built part of that codebase)	<code>NullCommandHandler</code>	Compensation failures alert, don't silently log-and-continue	2–3 wk
B9 — Community Trust	Attestation, honest consensus scoring, conflict detection, traditional-authority endorsement — 4th Trust Engine signal	Base44's domain/field model as spec	Base44's scoring code outright	Real authenticated actor on every review	2–3 wk
B10 — Inheritance & Customary Law	Death verification → beneficiary validation → share calculation → certificate	Base44's + Emergent's domain models as spec	Both apps' actual implementations	PII routes through B5's encryption from day one	4–5 wk
B11 — Economic / Billing	Atomic ledger, real invoicing, payment webhooks, consumes Survey's revenue-share events	Emergent's webhook signature code (audited correct); Base44's <code>RevenueTransaction</code> field model	Mock payment endpoint entirely; Base44's billing crash/race bugs	Balance mutation only via one atomic ledger function, DB-constrained non-negative	3 wk
B12 — Knowledge Graph	Graph projection off the event outbox across all contexts	New build	—	Read-only, rebuildable from the outbox at any time	3–4 wk
B13 — Security	Real fraud detection, security incidents, permission auditor reading actual RLS policy config , pen-test harness	Both apps' module lists as spec	Base44's fictional shadow-table permission auditor; Emergent's "R-2 CLOSED" self-cert pattern	Permission auditor fails loudly on any table with missing/permissive policy	2–3 wk

Stage	Scope	Reuse from prior builds	Discard	Security fix baked in	Effort
B14 — Operations	Job queue, backup/recovery testing, deployment observability	Both apps' job-type catalogs as spec	Base44's disabled-by-default automations; unconditional <code>seed_demo_data()</code> -style startup seeding	All seeding environment-gated, never runs against a populated/production DB automatically	2 wk

Backend total: ~31–37 weeks sequential (reduced from v3's ~33–40 by delegating auth issuance to Keycloak/Auth0).

3. FRONTEND build plan

Stage	Scope	Reuse	Discard	Security fix	Effort
F0 Shell	Next.js App Router scaffold, Tailwind+shadcn/ui theme, layout, generated API client, <code>ProtectedRoute</code> /middleware on every route	Base44's shadcn/ui set + layout pattern; Emergent's SDK discipline	Both apps' auth-context implementations; Base44's Vite config	Route-audit lint rule; auth via Keycloak/Auth0 SDK, no token in localStorage	1–2 wk
F1 Auth	Login/register via Keycloak/Auth0 hosted flow, real CSRF/state handling	Base44's role-selection UX	Emergent's stateless OAuth callback	CSRF-protected callback (handled by IdP SDK)	1 wk
F2 Registry + Spatial UI	Parcel CRUD, GIS map, polygon editor, overlap/duplicate warnings live during registration, bulk import	Base44's map/form components as reference	Naive CSV parser; client-side-only LGA bounds check	Spatial validation calls B4 server-side, never trusts client geometry	3–4 wk
F3 Evidence UI	Upload/list/detail/tabs	Emergent's evidence pages wholesale	—	Missing <code>ProtectedRoute</code> added; SDK path-param encoding fixed	2 wk
F3.5 Surveyor / Surveyor Network UI	Surveyor dashboard, network directory, public profile, archive import wizard	<code>SurveyorDashboard.jsx</code> , <code>SurveyorNetwork.jsx</code> , <code>SurveyorPublicProfile.jsx</code> , <code>ArchiveImportWizard.jsx</code> ported closely	Base44's separate legacy-tier survey pages, consolidated	Assignment actions require server-verified role/licence status	2–3 wk
F3.6 Trust & Transparency UI	Trust dashboard with explainable score breakdown, duplicate-alert dashboard, evidence timeline, public transparency portal	Base44's <code>TrustArchitecture.jsx</code> and <code>CommunityTransparency.jsx</code> layout	Base44's single-number trust score display with no factor breakdown	Every displayed score paired with its signal list	2–3 wk
F4 Community Trust UI	Attestation/review/consensus	Base44 UX as reference	Hardcoded- <code>"admin"</code> -actor components	Real authenticated actor on every review	2–3 wk
F5 Inheritance UI	Case/beneficiary/certificate mgmt	Base44 UX as reference	<code>document.write()</code> cert generator (confirmed XSS)	No client-built HTML strings anywhere	3–4 wk
F6 Economic UI	Wallet/billing/revenue, surveyor revenue-share view	Base44 pages as reference	Direct wallet/invoice writes	UI confirms only after atomic server write	2–3 wk
F7 Knowledge Graph explorer	Relationship visualization for fraud/due-diligence users	New build	—	Read-only viewer; no mutation surface via the graph	2–3 wk
F8 Security/Ops/Gov dashboards	Admin consoles	Both apps' layouts as reference	Score-without-evidence display pattern	Server-verified role, not sidebar-hiding	2–3 wk
F9 Public pages	Landing, verify, transparency	Both apps' content	—	CI route-audit confirms nothing else is public	1 wk

Frontend total: ~23–29 weeks.

4. FEATURE rollout timeline

Milestone	Unlocks	Needs	Cumulative (parallelized)
M0 Platform online	Auth works (via Keycloak/Auth0), CI green	B0–B2, F0–F1	Weeks 1–4
M1 Core registry MVP	Register/search/verify a parcel, real spatial validation at registration	B3–B4, F2, F9	Weeks 4–12
M2 Evidence-backed registry	Real evidence, integrity, enforced legal hold	B5, F3	Weeks 10–18
M2.4 Surveyor Network live	Surveyor onboarding, assignment, archive import, survey plans through the Evidence pipeline	B6, F3.5	Weeks 16–20
M2.5 Trust Intelligence	Confidence engine, duplicate detection, chain of custody, trust dashboard with explainable breakdown, 3 real signals	B7, F3.6	Weeks 18–24
M3 Community trust	Attestations feed Trust Engine as a 4th signal; consensus, traditional authority	B8–B9, F4	Weeks 22–28
M4 Inheritance & customary law	Death→beneficiary→share→certificate, PII-protected	B10, F5	Weeks 26–34
M5 Economic operations	Real billing, atomic wallets, surveyor revenue-share paid out	B11, F6	Weeks 30–36
M6 Knowledge Graph & fraud intelligence	Relationship graph live, fraud detection consumes it	B12–B13, F7–F8	Weeks 34–40
M7 Operations hardening & pilot launch	Real job queue, backup/recovery tested, environment-gated seeding, runbook	B14, F9 polish	Weeks 38–42

Total to pilot-ready: ~38–40 weeks.

5. Maturity read: Base44 vs. M2.4/M2.5 deliverables

Deliverable	Base44 state	Read
Surveyor Network / Dashboard / Archive Import (M2.4)	Fully scaffolded, routed, but 0 live records — never exercised	~70% reusable as UX/schema, ~0% backend logic to inherit
Confidence Engine	<code>lvEvidenceConfidence</code> / <code>lvAttestationConfidence</code> exist, inverted <code>risk_score</code> , unauthenticated write	~50% spec, 0% code
Duplicate Detection	GPS-proximity only, 7 redundant automations, 0 alerts ever generated	~40% spec
Chain of Custody	Functionally real (10 records confirmed working), <code>EvidenceChain</code> entity has no RLS at all	~55% UX, needs full security rebuild
Trust Dashboard	Real, well-built UI; underlying number is the confirmed "100/A_PLUS/GO despite 0 evidence" defect	~60% UI, ~10% trustworthy data
Transparency Dashboard	Confirmed working with real data (8 attestations)	~65% directly reusable
Explainable Trust Score	<code>subscores</code> breakdown shape exists structurally; every value fabricated by no-op branches	~15%

6. Open sub-decisions

- Keycloak vs. Auth0** — Keycloak recommended default (self-hosted, avoids per-MAU cost, better data-residency story for a land-registry pilot); Auth0 as a faster-to-bootstrap fallback. See [docs/adr/ADR-004-authentication-authorisation-model.md](#) .
- Knowledge Graph engine** — Neo4j (separate service, more capable) vs. Postgres + Apache AGE (one database, less mature tooling). Leaning AGE for a lean team.
- Monorepo vs. multi-repo** — monorepo recommended (`/frontend` , `/backend` , `/infra`). See [docs/adr/ADR-001-repository-strategy.md](#) .
- Team size — assumed 1–3 engineers + Claude Code; effort estimates rescale roughly linearly if that's wrong.

Phase Gates — Process & Quality Model

This document is the **process/governance layer**. It answers "how do we know work is done and safe to build on." For "what actually gets built, in what order," see [docs/REBUILD_PLAN.md](#) — the 13 bounded contexts and their B0–B14 / F0–F9 / M0–M7 sequencing.

Binding rule: Claude Code is not permitted to start a new production phase until the current phase has passed all mandatory quality gates. Every phase must end with successful architecture review, security review, automated testing, performance validation, documentation updates, and deployment readiness. If any gate fails, Claude must prioritize fixing the issue before implementing new features. **Progress is measured by verified quality, not by the number of completed features.**

Reconciliation: how this relates to the bounded-context plan

Three ways of slicing this project were on the table during planning: a generic 13-phase delivery lifecycle, a generic business-capability sprint list, and the audit-derived 13 bounded-context plan. The decision, recorded here so no future session re-litigates it:

- Phases (below) are the cross-cutting gate/process layer** — they define what "done" means at each stage of the whole platform's maturity, regardless of which bounded context is being worked on.
- The generic Sprint 1–8 business-capability list is retired.** It was a thinner subset of the bounded-context plan (no Spatial Intelligence, no Trust Engine, no Knowledge Graph, minimal Inheritance/Customary Law depth) and having three overlapping taxonomies risked confusion rather than clarity.
- Sprints = one per bounded context** (13 total, per [docs/REBUILD_PLAN.md](#) §1). Each sprint follows the Development Operating Model below and must pass the relevant Phase's gates before merge.

Mapping: which bounded-context stage satisfies which phase

Phase	Satisfied by
0 — Enterprise Planning	This document + docs/REBUILD_PLAN.md + the ADR set in docs/adr/
1 — System Architecture	docs/adr/ADR-001 through ADR-008
2 — Development Environment	New sub-stage B0.0 : repo scaffold, Docker Compose, Terraform baseline, env-var conventions, Keycloak/Auth0 tenant setup, secrets manager wiring
3 — Foundation	B0 (kernel) + B1–B2 (Identity/AuthZ) + F0 (frontend scaffold)
4 — Database	B0 kernel schema + B3 Registry schema (PostGIS setup, Alembic migrations)
5 — Core Services	B3 Registry, B4 Spatial Intelligence, B5 Evidence, B6 Survey, B8 Workflow, B12 Knowledge Graph (as a read-projection service)
6 — AI Layer	Scoped concretely in docs/adr/ADR-008-ai-integration-strategy.md : OCR/document classification (Evidence uploads), AI-assisted signals feeding the Trust Engine as advisory input only, never authoritative , fraud-pattern detection over the Knowledge Graph
7 — Payments	B11 Economic/Billing
8 — Security Hardening	B13 Security — a dedicated pre-launch sweep, distinct from the per-sprint SECURITY TEST loop step below
9 — Performance	Explicit checkpoint tied to B14 Operations + each context's own perf criteria in the DoD
10 — Production Readiness	B14 Operations (job queue, monitoring, backup/DR) + M7
11 — Launch	M7 (pilot-ready milestone)
12 — Growth	Beyond current plan scope — a v2 roadmap, new contexts, scaling work

Phase 0 — Enterprise Planning

Objective: Understand the business completely before writing code.

Deliverables: Business Vision · Product Vision · Stakeholders · Functional Requirements · Non-functional Requirements · Risk Analysis · Success Criteria

Exit criteria: - ✓ Vision approved - ✓ Requirements approved - ✓ Scope frozen

No code yet.

Phase 1 — System Architecture

Deliverables: C4 diagrams · Architecture diagrams · Technology selection · Repository strategy · Folder structure · Infrastructure · Security architecture

Checks: Architecture Review · Threat Model · Scalability Review · Performance Review · Cost Review

Exit criteria: No unresolved architecture issues.

Phase 2 — Development Environment

Deliverables: GitHub · Claude Code · VS Code · Docker · Terraform · FastAPI + Next.js scaffolds · Keycloak (or Auth0) · Paystack · Stripe · AWS/Azure Secrets Manager · Development / Staging / Production environments

Checks: Environment validation · Secrets validation · Git branching · CI/CD · Rollback test

Exit criteria: Three isolated environments operational.

Phase 3 — Foundation

Claude builds: Authentication (Keycloak/Auth0 integration) · Authorization (PDP/PEP/PIP) · Database · API · Logging · Monitoring · Audit

Checks: Authentication · OWASP · RLS · JWT (verification against IdP JWKS) · Logout · Permission testing · Session expiry · Rate limiting · Dependency audit

Exit criteria: Zero critical vulnerabilities.

Phase 4 — Database

Claude builds: Schema · Indexes · Relationships · Migrations (Alembic) · RLS · Audit logs

Checks: Performance · Query analysis · Load testing · Backup · Recovery · Data integrity

Exit criteria: Database production ready.

Phase 5 — Core Services

Claude builds: Property/Parcel Registry · Spatial Intelligence · Evidence · Survey · Search · Document Upload · Verification · Dashboard · Knowledge Graph projection

Checks: Integration testing · API testing · Performance · Edge cases · Validation

Exit criteria: 95% automated test coverage.

Phase 6 — AI Layer

Claude builds: Classification (Evidence OCR/document type detection) · Document AI · Verification-assisting AI signals (feeding the Trust Engine as advisory input) · Recommendations

Checks: Evaluation · Hallucination testing · Latency · Cost · Prompt injection · Fallback chain · Model routing

Exit criteria: AI quality above target threshold. AI output is never treated as authoritative — see [docs/adr/ADR-008-ai-integration-strategy.md](#).

Phase 7 — Payments

Claude builds: Paystack · Stripe · Invoices · Credits · Subscriptions

Checks: Sandbox testing · Webhook verification · Duplicate protection · Refund flow · Currency testing

Exit criteria: Successful end-to-end payment tests.

Phase 8 — Security Hardening

Claude executes: OWASP ZAP · Burp Suite · GitGuardian · Snyk · Secret scan · API penetration testing · Threat modelling

Checks: No SQL Injection · No XSS · No CSRF · No broken authentication · No privilege escalation

Exit criteria: No critical or high-severity findings.

Phase 9 — Performance

Claude performs: Load tests · Caching · Redis · CDN · Compression · Image optimization · Database tuning

Checks: Page load · API latency · Database latency · Memory · CPU · Cost

Exit criteria: Performance targets achieved.

Phase 10 — Production Readiness

Claude verifies: Environment variables · Secrets · Monitoring · Alerts · Logging · Backups · Recovery · Canary deployment · Rollback

Checks: Deployment rehearsal · Rollback rehearsal · Disaster recovery

Exit criteria: Go/No-Go decision.

Phase 11 — Launch

Production deployment · Monitoring · Analytics · Incident management · Daily health checks · Weekly review · Monthly audit

Phase 12 — Growth

AI optimisation · Infrastructure optimisation · Scaling · Government APIs · GIS · Machine Learning · Fraud Detection · Enterprise customers

The Claude Code Loop

Every sprint (= one bounded context) follows this cycle:



No shortcuts. The "AI REVIEW" step maps to this repo's `/code-review` and `/security-review` tooling — it is a real, run step, not aspirational text.

10 Standing Review Questions

Ask these at every gate check:

1. **Foundation:** Is the architecture still aligned with the business goals?
2. **Security:** Have OWASP Top 10 risks been checked?
3. **Performance:** Will this support (a) 10,000 users? (b) 100,000 users? (c) 1 million users?
4. **Cost:** Can infrastructure costs be reduced without sacrificing quality?
5. **Quality:** Has every feature been tested?
6. **Documentation:** Is documentation complete and up to date?
7. **Deployment:** Can this version be rolled back in under two minutes?
8. **Bugs:** Are there any known critical bugs?
9. **AI:** Has AI behaviour been evaluated?
10. **Future:** Will today's decisions block tomorrow's features?

Development Operating Model (per sprint = per bounded context)



Documentation and quality are part of development, not something left until the end.

Definition of Done (DoD)

Every feature, bounded context, sprint, and milestone must satisfy the same quality criteria before it is considered complete. Only then can Claude Code mark the task as complete. This document is invoked at three levels — see §4.

1. Tier 1 — Feature Done

A feature is only "Done" when:

Functional - Requirements implemented. - Acceptance criteria met. - Edge cases handled.

Code Quality - Passes linting (ESLint for TypeScript/Next.js, Ruff for Python/FastAPI). - Proper typing (TypeScript strict mode; Python type hints, checked with mypy). - No duplication. - Follows `docs/adr/` decisions and this repo's coding standards.

Testing - Unit tests pass. - Integration tests pass. - End-to-end tests pass where appropriate.

Security - Authorization verified (goes through the PDP/PEP — see `docs/adr/ADR-004-authentication-authorisation-model.md`, never a bespoke check). - Authentication verified. - Input validation complete. - Secrets not exposed. - OWASP checks passed. - **New entity/table? An RLS/authorization policy ships in the same commit — never a follow-up.** See `docs/ENGINEERING_RULES.md`.

Performance - Meets agreed response-time targets. - Database queries reviewed (no N+1, indexes checked against PostGIS/Postgres query plans where relevant). - No obvious bottlenecks.

Documentation - README updated if needed. - API documentation updated (OpenAPI schema regenerated). - Architecture updated if affected (`docs/REBUILD_PLAN.md`, relevant ADR). - Changelog updated.

Deployment - CI/CD pipeline passes. - Deployable to staging. - Rollback possible in under two minutes (see standing review question 7).

Non-negotiable, audit-evidenced criterion:

Every signal contributor to a computed score (Trust Engine, Confidence Engine, consensus calculation, any automated readiness/health score) **must have a test asserting zero-data or missing-data input yields a low or `INSUFFICIENT_DATA` result — never a passing score.**

This traces directly to the Base44 audit's confirmed finding: the platform's trust validation engine returned `100/A_PLUS/GO` while having zero evidence, zero certificates, and multiple disabled automations — because every one of its sub-checks silently treated "nothing to check" as "check passed." This must never recur here.

2. Tier 2 — Sprint (Bounded Context) Done

All planned features for the bounded context are integrated, tested, documented, and accepted:

- Every feature in the sprint meets Tier 1.
- Cross-context integration tested against every dependent context (per `docs/REBUILD_PLAN.md`'s stage dependency ordering).
- The relevant Phase gate(s) in `docs/PHASE_GATES.md` pass.

- Deployed to staging and demonstrable.
- DoD review completed and logged before merge.

3. Tier 3 — Product Done (MVP)

LandVault is considered MVP-complete when a user can:

1. Register and authenticate (via Keycloak/Auth0).
2. Search for a property.
3. Request property verification.
4. Upload supporting documents.
5. Pay using Paystack (Nigeria) or Stripe (international).
6. Receive verification status and notifications.
7. Access a secure document vault.
8. Use the platform from desktop and mobile.

Bounded-context coverage of this MVP list: Identity (1), Registry + Spatial Intelligence (2), Workflow + Community Trust in a minimal form (3), Evidence (4, 7), Economic/Billing (5), Workflow/notifications (6), Frontend responsive design (8).

Explicitly NOT required for this MVP definition: Trust Engine's full multi-signal explainable scoring (M2.5), Knowledge Graph, and the deep Inheritance & Customary Law scope (death verification, family ownership chains, regime-based share calculation). These are real, substantial parts of the domain — confirmed by the audit to represent 8+ dedicated entities and significant UI in the Base44 build — and are deliberately sequenced after MVP, not silently dropped. **This scope-down requires the Operator's explicit sign-off; record any change to it as a Ratification Log entry or ADR amendment, not a silent edit to this file.**

At MVP, you have a real product, not a prototype.

4. Where this gate is invoked

- **Per PR** — Tier 1, mechanically checked by CI.
- **Per sprint (bounded context)** — Tier 2, human-reviewed before merge to main.
- **Per milestone (M0–M7 in `docs/REBUILD_PLAN.md`)** — rolls up to Tier 3 once all MVP-covering contexts are Sprint Done.

5. Non-negotiable blockers (never waivable)

- No entity/table without an RLS/authorization policy.
- No permissive fallback default on a security-relevant env var (CORS origin, signing secret, rate-limit toggle).
- No second/parallel authorization path outside the PDP/PEP.
- No scoring/validation function that can report a pass on zero/missing data.
- No feature marked complete without an *observed* passing test run.

Engineering Operating Rules

This document defines how Claude Code works on this repository. Without these rules, every session starts making its own assumptions. `CLAUDE.md` restates the most load-bearing rules inline for every session; **this document is authoritative if the two ever diverge.**

Every rule below traces to a specific, confirmed defect found in the `landsecure-registry` (Base44) or `landverify-nigeria-101-NEW` (Emergent) audits — see `docs/audits/`. These aren't hypothetical concerns; they are the exact mechanisms that produced the worst findings in both prior builds of this product.

1. Authorization

Rule: No new entity/table ships without an RLS or PDP/PEP authorization policy in the *same commit* that creates it.

Why: Base44's `OrganizationWallet`, `ServiceRequest`, and `Invoice` entities all shipped with `update: {}` — an unconditional public-write policy — enabling any authenticated user to edit any organization's wallet balance or mark any invoice paid. Separately, 15 entities (including the entire inheritance/death-verification module) had **no RLS block at all**. Neither gap was ever caught because policy authoring was treated as a follow-up task, not part of the entity's definition.

Rule: There is exactly one authorization path: the PDP/PEP/PIP engine. No parallel or "legacy" auth system is ever introduced, for any reason, even temporarily.

Why: The Emergent build's audit found a fully separate, undocumented session-cookie RBAC system (`core/security.py`) running alongside its otherwise well-designed PDP/PEP engine, plus a `dev-login` endpoint with no environment gate at all that minted valid admin sessions to anyone. Two authorization systems means two places to get security right — this repo will only ever have one.

2. Secure-by-default configuration

Rule: No security-relevant environment variable may have a permissive fallback default. If it's unset, the application must fail to start, not silently degrade to an insecure default.

Why: Emergent's CORS origin defaulted to `"*"` combined with `allow_credentials=True` — meaning any website could make credentialed cross-origin requests unless an operator remembered to set `CORS_ORIGINS`. Its certificate-transparency-log signing key fell back to the literal string `"dev-signing-secret"` if unset. Both were exploitable purely by omission, not by any attacker action.

3. Scoring and validation honesty

Rule: Every automated scoring, validation, or health-check function must fail safe: missing or zero data yields a low/neutral/ `INSUFFICIENT_DATA` result, never a passing score. See `docs/DOD.md` §1 for the corresponding test requirement.

Why: Both audited builds had this exact defect independently. Base44's trust validation engine reported `100/A_PLUS/GO` with zero real evidence because its sub-checks used `if (condition) { passed++ } else { passed++ }` — incrementing "passed" regardless of outcome. Its own permission auditor validated a hardcoded internal table against itself rather than reading real policy config, so it could never detect the RLS gaps above. This is not a one-off bug pattern to fix once; it's a category this repo tests against by rule.

4. When Claude may act autonomously vs. must stop and ask

Claude may proceed without asking when: - The change is in-scope of the current bounded context/sprint. - Tests are green before and after the change. - No database schema change is involved. - No new external dependency is introduced.

Claude must stop and ask a human when: - The change crosses bounded-context boundaries. - A database schema or migration change is involved. - A new external dependency would be introduced. - The change touches authentication, authorization, payments, or evidence integrity. - Any data deletion is involved. - Requirements are ambiguous or conflict with an existing ADR. - Anything irreversible in production is about to happen. - The work is legal/compliance-adjacent (e.g. customary-law share-calculation logic, death-verification handling).

5. Dependencies

Rule: Adding a new dependency requires explicit human approval, a justification in the PR description, and a pinned version.

Why: Both audited codebases accumulated dependencies without a consistent review discipline (mixed SDK versions calling the same backend inconsistently in Emergent; an `emergentintegrations` /vendor-specific package pulled from a non-PyPI URL). Unreviewed dependencies are both a security surface and a lock-in risk.

6. Schema changes

Rule: Migrations must be reversible. A schema change ships with its matching RLS/policy update in the same commit (see §1). Rollback is tested in staging before the change reaches production — this is what standing review question 7 ("can this be rolled back in under two minutes?") is actually checking.

7. Never mark something complete without observing it pass

Rule: A feature, automation, or gate is only "complete" after Claude has actually run the relevant test/check and observed it pass — not inferred from code inspection alone.

Why: This is the single rule that would have caught the largest share of both audits' findings. Both prior builds had features self-reported as "CONNECTED" or "PASSED" in their own dashboards that were confirmed, on live testing, to be broken, insecure, or fabricating their output entirely. Static confidence is not evidence.

8. Commit/PR discipline

- One bounded context per PR where feasible.
- The DoD checklist (`docs/DOD.md`) is required in the PR description.
- Git hooks are never skipped (`--no-verify` is not used without explicit human authorization for that specific instance).
- Commit messages describe *why*, not just *what*.

ADR-001 — Repository Strategy

Status: Accepted **Date:** 2026-07-13

Context

Two prior implementations of this product exist: `landsecure-registry` (a Base44-platform build, fully audited and found to have severe security/correctness defects across its RLS configuration, client-side-only authorization, and backend function logic) and `landverify-nigeria-101-NEW` (an Emergent-platform build, more rigorously architected but undermined by a parallel legacy auth system and several critical findings of its own). Both audits are preserved in `docs/audits/`.

We need to decide where the rebuild lives, and how the frontend, backend, and infrastructure code are organized relative to each other.

Decision

1. **A fresh repository** (`aquasavannah-landvault`) hosts the rebuild. `landsecure-registry` is left untouched as a historical archive — not merged, not migrated, not deleted. Its audit reports are copied (not moved) into `docs/audits/` here for continuity.
2. **Monorepo structure**, not separate repos per layer: `/frontend — Next.js + TypeScript` `/backend — Python + FastAPI`, `DDD/hexagonal bounded contexts` `/infra — Terraform, Docker` `/docs — this planning package`

Consequences

- A small team (or a single Claude-Code-driven contributor) can reason across the full stack in one session and one PR when a change genuinely spans frontend/backend (e.g. a new API contract field).
- One CI pipeline, one issue tracker, one source of truth for the whole product.

- Trade-off: a monorepo requires discipline to keep bounded-context boundaries real inside `/backend` rather than letting them blur just because they're in the same repo — this is why `docs/ENGINEERING_RULES.md` §4 requires explicit approval to cross bounded-context boundaries regardless of physical repo layout.
- `landsecure-registry` remaining untouched means its GitHub remote, CI workflows (including the previously-confirmed-broken `npm test` / `npm publish` pipeline), and Base44 auto-sync behavior are irrelevant to this repo and require no cleanup here.

ADR-002 — System Architecture

Status: Accepted **Date:** 2026-07-13

Context

`landverify-nigeria-101-NEW`'s audit found its Domain-Driven Design / hexagonal architecture (domain → ports → adapters → application → api layering, bounded contexts, event-sourced transactional outbox) to be genuinely well-built where it wasn't undermined by a parallel legacy system. `landsecure-registry`'s audit found the opposite problem in a different form: no architectural boundaries at all — a flat collection of ~60 serverless functions and a React SPA calling entities directly, with business logic, authorization, and data access all intermixed.

Decision

Adopt DDD/hexagonal architecture with **13 bounded contexts** (full detail and dependency ordering in `docs/REBUILD_PLAN.md` §1):

Identity · Registry · Spatial Intelligence · Evidence · Survey · Trust Engine · Workflow · Community Trust · Inheritance & Customary Law · Economic/Billing · Knowledge Graph · Security · Operations

Each context follows the same internal layering: - **domain/** — pure aggregates, invariants, value objects, events. No I/O. - **ports/** — repository and specification interfaces. - **adapters/** — concrete Postgres/S3/external-API implementations of those ports. - **application/** — orchestrating services: open a transaction, drive aggregate commands, persist, publish events. - **api/** — FastAPI routers + strictly-typed request/response DTOs (no mass-assignment — extra fields rejected).

Cross-context communication happens **only** through the transactional event outbox (see ADR-007) — no context reaches into another's tables directly. Knowledge Graph (ADR-005/REBUILD_PLAN §1) and Trust Engine are both built as event-subscribing read-projections of this outbox, not as contexts with their own authoritative writes into other contexts' data.

Consequences

- Aggregate invariants (immutable fields, monotonic versions, append-only history) are enforced once, in the domain layer, not re-implemented per caller.
- New signal sources (a new bounded context) can feed the Trust Engine or Knowledge Graph by adding an event subscriber, without modifying either engine's core — this is the direct fix for Base44's "trust score bolted on top of 8 already-built silos" failure mode.
- Trade-off: this is more upfront structure than either prior build had (Base44 had none; Emergent had it for 4 of what are now 13 contexts). The 13-context scope is deliberately larger than either prior attempt — see `docs/REBUILD_PLAN.md` §4 for the resulting ~38–40 week timeline, and `docs/DOD.md` §3 for which contexts are in vs. out of MVP scope.

ADR-003 — Database Choice

Status: Accepted **Date:** 2026-07-13

Context

Nearly every critical bug found across both audits traced back to the same root cause: no real ACID transactions and no native row-level security.

- Base44 (a document-store-backed low-code platform): `CreditWallet`'s update policy allowed the record owner to edit their own `credit_balance` field directly (record-level RLS, not field-level) — a direct self-service financial-fraud vector. `OrganizationWallet`, `ServiceRequest`, and `Invoice` had `update: {}` — unconditionally public. 15 entities had no RLS block at all.
- Emergent (MongoDB via Motor): a standalone (non-replica-set) MongoDB deployment silently falls back to no-transaction mode, so a crash between a wallet `$inc` and its corresponding transaction-log insert leaves no idempotency record — a webhook retry can re-credit the same purchase. `lvServiceBilling`'s credit-reservation flow crashed on an undefined variable *after* committing a partial write, leaving reserved credits permanently locked with no compensating transaction.

Decision

PostgreSQL, self-managed (not a managed document store, not Supabase-managed) for Identity role/tenant mapping, Registry, Spatial Intelligence (with **PostGIS**), Survey, and Economic/Billing.

- Real multi-statement ACID transactions for every money-moving or state-transition operation (see ADR-006).
- **Native row-level security policies**, enforced at the database engine — not an application-layer convention that's easy to forget or bypass by using a different code path (both audits found exactly this "opt-in and forgettable" pattern repeated across dozens of call sites).
- PostGIS for genuine polygon overlap, adjacency, and spatial-index queries (see `docs/REBUILD_PLAN.md` B4) — the audits found both prior builds' "duplicate detection" was GPS-proximity-distance-only, never real geometry containment/overlap.
- Alembic for migrations, with the rule (see `docs/ENGINEERING_RULES.md` §6) that every migration ships reversible and with its RLS policy update in the same commit.

Evidence binaries remain in S3-compatible object storage (never in Postgres) — see ADR-007. Knowledge Graph (ADR-002) uses a dedicated graph store, fed by projection off the event outbox, never Postgres directly for its traversal queries.

Consequences

- Structurally closes the entire "non-atomic financial operation" and "forgot the RLS policy" bug classes rather than relying on developer discipline per call site.
- Requires an actual migration/ops discipline (Alembic, connection pooling, PostGIS extension management) that a managed platform would otherwise absorb — accepted as the cost of the control this rebuild needs, particularly for money and land-title data.

- Self-managed (not Supabase) per the Operator's explicit instruction that Supabase not become the entire architecture; Supabase Auth remains a documented fallback option for identity only (ADR-004), not for the database.

ADR-004 — Authentication & Authorisation Model

Status: Accepted (Keycloak vs. Auth0 left as an explicit open sub-decision — see below) **Date:** 2026-07-13

Context

The Emergent audit's single most consequential finding was architectural, not a line-level bug: a fully separate, undocumented session-cookie RBAC system (`core/security.py`) ran in parallel with an otherwise well-designed PDP/PEP/PIP authorization engine, and neither the security contract (`contracts/v1/security/role_matrix.json`) nor any test suite covered the legacy path. On top of that, a `dev-login` endpoint had **no environment gate at all** and minted valid admin sessions to any unauthenticated caller; `ENABLE_TEST_ENDPOINTS` defaulted to `true` rather than `false`; and `assign_role` had no check preventing a `compliance_officer` from promoting themselves to `super_admin`.

Base44 had no backend authorization at all — every "who can approve this" check was a client-side `if (user.role === ...)` with the corresponding backend entity either unprotected or protected by RLS with the same class of gaps described in ADR-003.

The PDP/PEP/PIP design itself — a Policy Decision Point evaluating registered policies per bounded context, a Policy Enforcement Point wrapping every route, a Policy Information Point supplying attributes — was audited as **structurally sound and fail-closed by design** (an unmatched or erroring policy denies, never permits). The defect was letting a second system exist beside it, and a handful of specific implementation gaps (rate limiting defaulted off in the actual app wiring despite the module defaulting on; tenant/country isolation policies silently no-opping when a resource descriptor omitted `tenant_id`; a 24-hour compromised-key grace window with no hard-revoke).

Decision

1. **Authentication (who are you) is delegated to an external IdP — Keycloak** (self-hosted, open-source) as the default, with **Auth0** as a faster-to-bootstrap alternative if the team wants a managed option initially. This removes an entire class of custom-auth bugs (password hashing, MFA, OAuth flow correctness, JWT signing-key issuance and rotation) by not reimplementing them. - **Open sub-decision:** Keycloak vs. Auth0 is not yet finalized. Recommendation: Keycloak, because a land-registry platform handling government/citizen PII benefits from self-hosted data residency and avoids per-monthly-active-user pricing at pilot scale; Auth0 remains the fallback if faster initial setup outweighs those concerns. This ADR will be updated (not silently edited — see the Governance Amendment Procedure in `docs/PHASE_GATES.md`) once decided.
2. **Authorization (what can you do) — the PDP/PEP/PIP engine is retained in full**, ported from the Emergent design: - The PEP verifies every incoming request's JWT against the IdP's JWKS endpoint (not a self-issued KeyStore — Keycloak/Auth0 own key rotation now). - The PDP evaluates registered per-bounded-context policies (see each context's `authorization.py` -equivalent) and **fails closed**: no matching policy, an error, or ambiguity all result in denial, never permission. - Tenant and country scoping is enforced at **two independent layers** — the PDP resource descriptor AND the Postgres repository/RLS layer (ADR-003) — so a caller who forgets to populate one still cannot bypass the other.
3. **There is exactly one authorization path.** No session-cookie fallback, no `dev-login`, no test-mode-by-default endpoint ever ships. Test/seed data creation is always explicitly environment-gated (see `docs/ENGINEERING_RULES.md` §2 and §7).
4. Role assignment requires an explicit **hierarchy check**: a principal can never grant a role higher than their own, and never to themselves for the purpose of elevation. This is a direct fix for the confirmed Emergent `assign_role` privilege-escalation path.
5. Rate limiting is on by default with no override that silently disables it in production; the client-IP source used for rate limiting and audit logging is only trusted behind a configured reverse-proxy allowlist, never a raw `X-Forwarded-For` header.

Consequences

- Significantly less custom authentication code to write and maintain than either prior build attempted (see `docs/REBUILD_PLAN.md` B1/B2 effort reduction).
- The PDP/PEP engine remains the one piece of genuine, reusable intellectual property from the Emergent codebase — its design survives the rebuild even though almost none of its literal code does (JWT issuance/KeyStore code is now Discard; PDP/PEP/PIP decision logic is Reuse).
- Requires role/tenant/permission data to be maintained in our own Postgres, joined to the IdP's stable subject (`sub`) claim — an explicit synchronization point (first-login provisioning or IdP webhook) that must itself be covered by the DoD's "every new entity ships with an authorization policy" rule.

ADR-005 — Property Registry Data Model

Status: Accepted **Date:** 2026-07-13

Context

Both prior builds modeled the land parcel differently, and neither got it fully right. Base44 ran **two parallel parcel entities** (`LandParcel`, a legacy model with 500+ historical records, and `LandVaultParcel`, a newer model with essentially no live data) with a confusing, never-completed migration between them, and no single authoritative record. Emergent's Registry bounded context had a single, well-designed `LandVault` aggregate — the audit confirmed its immutable-field invariants (`registry_id`, `parcel_number`, `tenant_id`, `country_code`, `created_at`, `origin`) and atomic parcel-number allocator (a genuine single `find_one_and_update` with `$inc` /upsert, zero duplicates under concurrent load) were sound — but its "owner-only" update authorization was dead code in practice: the PDP resource descriptor never carried `created_by`, so any principal holding a create-tier role could update location/geometry/ownership on *any* parcel in their tenant, not just ones they created or were assigned to. Its migration tool's idempotency guarantee also didn't hold under concurrent runs (the provenance index used to detect already-imported records wasn't unique), and unmappable geometry was silently dropped rather than quarantined as its own documentation claimed.

Decision

One canonical parcel aggregate — `LandVault` — per the Emergent design, carried into this rebuild's Registry bounded context (`docs/REBUILD_PLAN.md` B3):

- Immutable at creation: `registry_id`, `parcel_number` (from the atomic allocator, reused pattern), `tenant_id`, `country_code`, `created_at`, `origin`.

- Mutable via explicit, invariant-guarded commands only: `UpdateLocation`, `UpdateGeometry` (delegates to Spatial Intelligence, ADR per `docs/REBUILD_PLAN.md` B4), `UpdateOwnershipContact`, `RecordOwnershipTransfer` (append-only `ownership_history[]`), `UpdateSurvey`, `UpdateCommunityData`, `Archive` (one-way).
- **Fixed:** ownership-transfer and update authorization checks the real actor identity against the resource (`created_by` / assigned surveyor / field agent), not just role membership — the PDP resource descriptor is populated correctly this time, and the aggregate itself independently re-checks actor identity as true defense-in-depth (not the documented-but-absent version from the audit).
- **Fixed:** the legacy-import migration tool's provenance-uniqueness constraint is enforced at the database level (a real unique index, not just an application-level pre-check), and unmappable geometry is quarantined, never silently dropped.
- Base44's field-level richness (the union of fields across `LandParcel` and `LandVaultParcel`) is used as the requirements checklist for what this one aggregate needs to capture — not its code, which is discarded.

There is **one** parcel entity in this rebuild. No legacy/new dual-track pattern.

Consequences

- Eliminates the "which parcel record is authoritative" confusion that existed in Base44 and the dual-write drift risk that existed in Emergent's legacy-compatibility adapter (a non-transactional mirror write with no reconciliation path, confirmed in the audit to be able to leave a parcel durably registered but invisible to legacy readers).
- Ownership-transfer and update operations are meaningfully restricted per actor, not just per role — closing a confirmed authorization gap.
- The atomic allocator and immutable-field invariant logic are reused near-verbatim from Emergent, since the audit found no defect in that specific logic.

ADR-006 — Payment Architecture (Paystack + Stripe)

Status: Accepted Date: 2026-07-13

Context

Both audits found serious payment/credit-ledger defects:

- Base44's `lvServiceBilling` function reserved credits atomically, then created a `ServiceRequest` record, then referenced an **undefined variable** while building its audit-log entry — crashing *after* both prior writes had already committed. The client received a 500 error and never learned the reservation succeeded, so it could never call `complete` / `fail` to release it — a permanent, unbounded credit leak on every failed attempt. Separately, `CreditWallet`'s RLS allowed the record owner to edit their own balance directly, and `OrganizationWallet` / `ServiceRequest` / `Invoice` allowed *any* authenticated user to edit *anyone's* wallet, service request, or invoice.
- Emergent's webhook signature verification (Paystack HMAC-SHA-512 via `hmac.compare_digest`, Stripe via the official SDK's `Webhook.construct_event`) was confirmed **correct** — raw request body used for HMAC, constant-time comparison, correct secret. However, an unauthenticated mock-payment webhook (`POST /api/webhook/mock`) was always mounted regardless of configured provider, letting a user create a real checkout to obtain a legitimate `reference`, never pay, then call the mock endpoint directly to receive real credits for free. A standalone (non-replica-set) MongoDB deployment also silently broke transactional atomicity between the wallet balance update and the transaction-log insert, risking double-crediting on webhook retry.

Decision

1. **Paystack** (Nigeria-first) and **Stripe** (diaspora/international) as the two payment providers, per the Operator's existing choice.
2. **Reuse Emergent's webhook signature-verification code near-verbatim** — it was audited as cryptographically sound and is not being rewritten from scratch.
3. **No mock/test payment endpoint is ever compiled into a non-development build.** Test payment flows exist only behind a build-time flag that cannot be enabled in staging or production images (not merely an environment-variable check at runtime, which is what failed in the Emergent audit).
4. **All balance-affecting operations go through one atomic ledger function**, backed by a real Postgres transaction (ADR-003) — never a check-then-`$inc` -separately pattern. The database enforces a non-negative balance constraint; the application layer cannot bypass it.
5. **Idempotency keys** on every purchase/consume/refund operation, so a webhook retry or duplicate client request cannot double-apply.
6. Invoices auto-generate on service completion — no admin-gated dead end (Base44's `lvInvoiceGenerator` required admin auth to invoke, but nothing auto-triggered it after service completion, leaving `₦25,000` in confirmed-but-unbilled revenue in the audited data).

Consequences

- Structurally eliminates the "phantom reservation" and "self-service balance edit" bug classes found in Base44, and the "always-on mock webhook" and "standalone-Mongo breaks atomicity" bugs found in Emergent (moot here since we use Postgres, ADR-003).
- Webhook handler code ports with high confidence directly from Emergent, saving implementation time relative to writing this from scratch.
- Requires the ledger function to be the *only* write path to wallet/invoice/service-request tables — enforced by the RLS/authorization policy required on every entity (`docs/ENGINEERING_RULES.md` §1), not by convention alone.

ADR-007 — Audit Trail & Evidence Model

Status: Accepted Date: 2026-07-13

Context

Land-title evidence is the core trust proposition of this product, so its integrity guarantees have to be real, not aspirational. The Emergent audit found the *design* of its evidence pipeline sound — server-side hashing with an independent streamed read-back re-hash (no trust placed in any client-supplied hash claim), Merkle-tree batch anchoring to an internal certificate-transparency-style log plus OpenTimestamps as an independent secondary anchor, chain-of-custody and legal-hold as append-only chained aggregates — but found the *implementation* of "immutability" was fake at the storage layer: the only wired storage adapter was local-filesystem with a `chmod 0o400` "lock" (meaningless if the process runs as root, which is typical in containers) defaulting to `/tmp`, which most container platforms wipe on every restart; the real S3/R2 adapter was never implemented beyond a stub. Legal hold was a database record with no enforcement anywhere — the domain code's own docstring

admitted enforcement was deferred to a "retention sweeper" that was never built, so an active legal hold currently blocks nothing. Two separate "integrity check" functions similarly checked a status flag rather than recomputing and comparing any actual hash — a pattern of security theater that recurred three separate times in that codebase.

Separately, both audits found the underlying audit-log mechanism itself (an append-only, hash-chained log of every domain event) to be a correct idea let down by one gap: no function anywhere actually *verified* the hash chain for tampering. A tamper-evident log nobody ever checks is not meaningfully different from a log that isn't tamper-evident.

Decision

1. **Event sourcing with a transactional outbox** at the kernel level ([docs/REBUILD_PLAN.md](#) B0) — every domain event across every bounded context is written atomically with the state change that produced it, then published for downstream subscribers (Trust Engine, Knowledge Graph, notifications). This is Emergent's outbox design, reused, with its one gap closed: a real `verify_chain()` integrity checker ships as part of B1, run on a schedule, not left as a theoretical guarantee.
2. **Evidence storage is real S3-compatible object storage with Object Lock in Compliance mode from day one** — never local filesystem, never a chmod-based convention. The R2/S3 adapter is built *first*, not deferred, precisely because the audit showed what happens when it's left for later (it never gets built).
3. **Legal hold is enforced as a guard checked at every delete/archive/seal-release code path**, implemented once as a shared mechanism every relevant command calls through — not a record any individual code path might forget to check.
4. **Hash/integrity verification actually recomputes and compares hashes**. No function reports "valid" based on a status flag or field-presence check alone.
5. Break-glass cross-tenant/cross-country evidence access requires the dual-authorization it's documented to require, mechanically enforced (rejected if the second approver is absent), and the security-incident record is written durably *before* the unwrap operation returns.

Consequences

- The evidence pipeline's hashing/verification/Merkle-anchoring *logic* ports from Emergent with high confidence (audited sound); the storage adapter and legal-hold enforcement are new work, not a port, because the audit confirmed neither existed in a real form.
- This is the single most implementation-heavy bounded context in the plan ([docs/REBUILD_PLAN.md](#) B5, 4–6 weeks) precisely because "real" WORM storage and enforced legal hold are non-trivial to get right — treated as core product risk, not a corner to cut for velocity.
- Every later context that needs tamper-evident records (Survey documents, Community attestations, Inheritance certificates) routes through this same pipeline rather than each building its own weaker version.

ADR-008 — AI Integration Strategy

Status: Accepted (scope may narrow further at Phase 6 kickoff — see [docs/PHASE_GATES.md](#)) **Date:** 2026-07-13

Context

Neither prior build implemented a real AI layer. Base44's job-queue inventory explicitly listed OCR processing as a defined job type with **no implementation behind it** ("OCR Jobs — MISSING, no function"). Its various "trust validation," "fraud detection," and "readiness scoring" functions were rule-based, not AI/ML, and — per ADR-002/ [docs/DOD.md](#) 's non-negotiable scoring-honesty rule — several of them were confirmed to be actively dishonest (reporting a perfect score regardless of input), which is a cautionary example for *any* automated scoring this rebuild adds, AI-based or not.

Emergent's own Foundation Specification (an internal governance document found during the audit) already stated an immutable design principle directly relevant here: **"AI is advisory only, never authoritative."** The audit found no violation of that principle, because no AI feature had been built yet to violate it — but the principle itself is sound and worth carrying forward explicitly rather than rediscovering.

Decision

Scope AI Integration concretely, rather than leaving Phase 6 generic:

1. **OCR / document classification** for Evidence uploads (survey plans, family agreements, death certificates, court orders) — extracting structured fields and a document-type classification to assist (not replace) human/community verification.
2. **AI-assisted signals feeding the Trust Engine as advisory input only, never authoritative**. The Trust Engine's score remains an explainable, rule-based aggregation of real signals (Evidence integrity, Spatial duplicate-geometry, Survey completion, Community consensus — see [docs/REBUILD_PLAN.md](#) B7); any AI-derived signal is one more named, weighted, *inspectable* input to that aggregation, never a black-box override of it. This directly enforces the non-negotiable rule in [docs/DOD.md](#) and [docs/ENGINEERING_RULES.md](#) §3 that no automated score may report a false pass — an opaque AI score would be impossible to test against that rule, so it is architecturally excluded from being authoritative.
3. **Fraud-pattern detection over the Knowledge Graph** (ADR-002) — relationship-based anomaly signals (e.g. one surveyor attesting an implausible number of unrelated parcels in a short window) surfaced to the Security bounded context ([docs/REBUILD_PLAN.md](#) B13) as investigation leads, not automatic actions.
4. **Explicit non-goals for MVP:** no AI-driven automated approval/rejection of any parcel, evidence item, or attestation; no AI feature ships without the Phase 6 checks in [docs/PHASE_GATES.md](#) (evaluation, hallucination testing, latency, cost, prompt-injection testing, a defined fallback chain, and model routing) passing first.

Consequences

- Phase 6 in [docs/PHASE_GATES.md](#) has a concrete owner instead of remaining an unscoped placeholder that the 16-step Claude Code Loop's "AI REVIEW" step would otherwise have nothing real to review.
- Every AI feature is additive to, never a replacement for, the explainable rule-based Trust Engine — preserving the "not AI magic" requirement from the Operator's own framing of the Trust Engine concept.
- Any future proposal to make an AI signal authoritative (e.g. auto-approving evidence) requires a superseding ADR, not a quiet change to the Trust Engine's aggregation weights.